گزارش پروژه طبقه‌بندی اصالت اسکناس با استفاده از ویژگی‌های تصویر

تشخیص اسکناس اصل از تقلبی با الگوریتم‌های K-Nearest Neighbors و Logistic Regression — نسخه‌ی سوم، بر اساس اجرای واقعی `notebook_full-7.ipynb` و `کد معمار (3).ipynb`

> **خلاصه‌ی تغییرات نسبت به نسخه‌ی قبلی گزارش:** در نسخه‌ی جدید کد، روش تقسیم داده به Train/Validation/Test اصلاح شد تا ردیف‌های کاملاً تکراری همیشه در یک بخش (Train یا Validation یا Test) باقی بمانند (Group-based Split)، و یک آزمایش صریح برای بررسی اثر Scale روی KNN اضافه شد. این تغییرات روش‌شناسی درست‌تر هستند، اما باعث شدند دقت نهایی هر دو مدل نسبت به گزارش قبلی (که به‌دلیل نشت داده از ردیف‌های تکراری کمی خوش‌بینانه بود) واقعی‌تر و کمی پایین‌تر گزارش شود. جزئیات کامل در بخش‌های ۴، ۶ و ۹ آمده است.

۱. معرفی پروژه و دیتاست

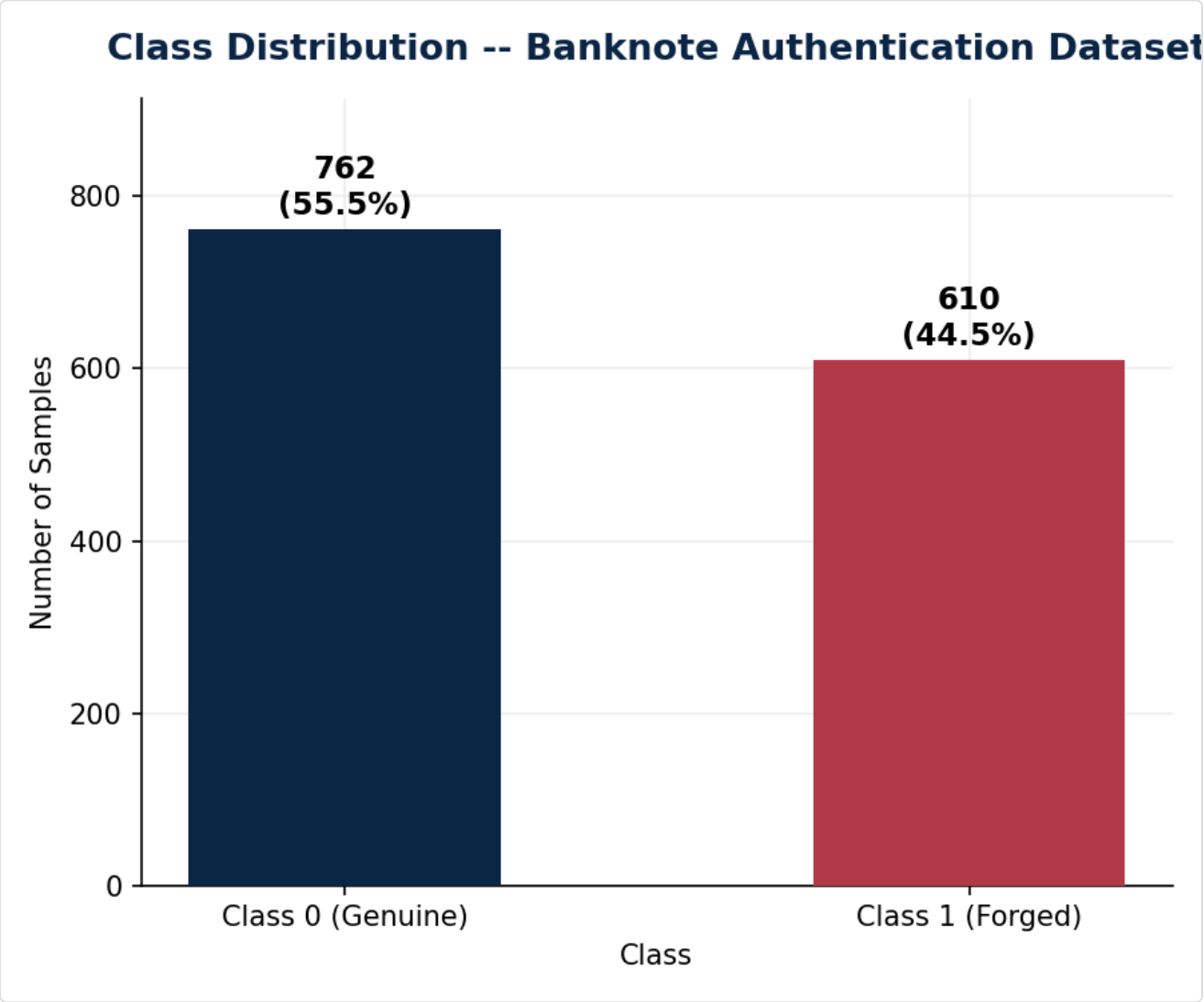
هدف این پروژه، طراحی یک مدل طبقه‌بندی برای تفکیک اسکناس‌های **اصل (Class 0)** از **تقلبی (Class 1)** است. ورودی مدل، تصویر مستقیم اسکناس نیست، بلکه چهار ویژگی عددی است که با تبدیل **Wavelet** از روی تصویر اسکناس استخراج شده‌اند. دیتاست مورد استفاده `BankNote_Authentication.csv` شامل **۱۳۷۲ نمونه** و ستون‌های زیر است:

ستون	توضیح
`variance`	واریانس تصویر تبدیل‌یافته با Wavelet
`skewness`	چولگی (نامتقارنی) توزیع تصویر
`kurtosis`	کشیدگی توزیع تصویر
`entropy`	آنتروپی (میزان بی‌نظمی) تصویر
`class`	برچسب هدف — ۰: اصل، ۱: تقلبی

۲. بررسی کیفیت داده (نوت‌بوک EDA)

محتوای نوت‌بوک EDA (`notebook_full-7.ipynb`) نسبت به نسخه‌ی قبلی بدون تغییر ماند (فقط مسیر بارگذاری فایل به‌روزرسانی شد)؛ نتایج زیر عیناً تکرارپذیرند:

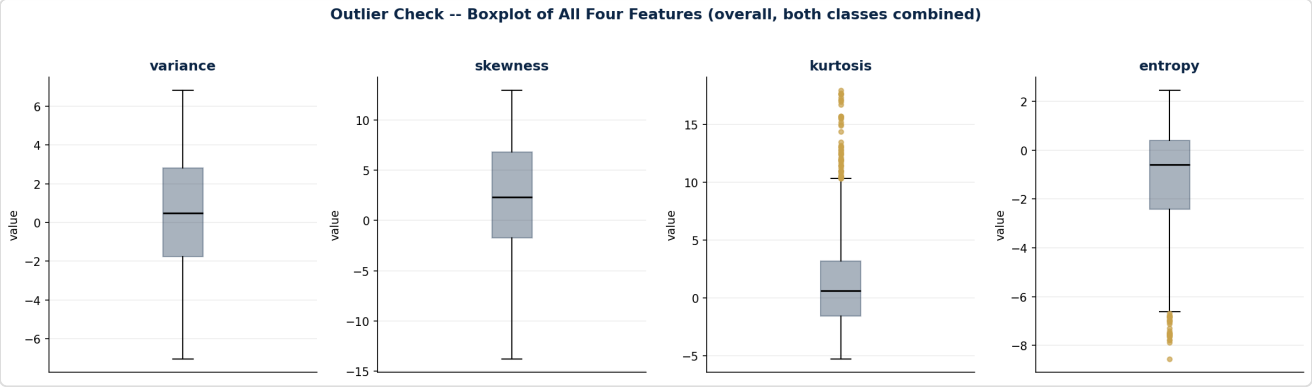
- **مقادیر گمشده:** صفر در همه‌ی ستون‌ها.
- **مقادیر غیرعددی:** صفر؛ همه‌ی ستون‌ها از نوع float64.
- **ردیف‌های تکراری:** ۳۵ ردیف در تکرار شرکت دارند و شامل ۱۱ الگوی متمایز هستند (هر الگو ۲ تا ۴ بار تکرار شده)، همگی متعلق به کلاس ۰. این نکته مستقیماً در بخش ۴ روی نحوه‌ی تقسیم داده اثر گذاشت.
- **توزیع کلاس‌ها:** کلاس ۰ دارای ۷۶۲ نمونه (۵۵.۵٪) و کلاس ۱ دارای ۶۱۰ نمونه (۴۴.۵٪).



نمودار ۱ — توزیع تعداد نمونه‌ها در دو کلاس (اصل / تقلبی)

مقادیر پرت (Outliers) — روش IQR

ویژگی	Q1	Q3	IQR	حد پایین	حد بالا	تعداد پرت	درصد پرت
variance	1.773-	2.821	4.594	8.665-	9.713	0	0.00٪
skewness	1.708-	6.815	8.523	14.492-	19.599	0	0.00٪
kurtosis	1.575-	3.179	4.754	8.706-	10.311	59	4.30٪
entropy	2.413-	0.395	2.808	6.626-	4.607	33	2.41٪



نمودار ۲ — Boxplot کلی هر چهار ویژگی برای بررسی نقاط پرت

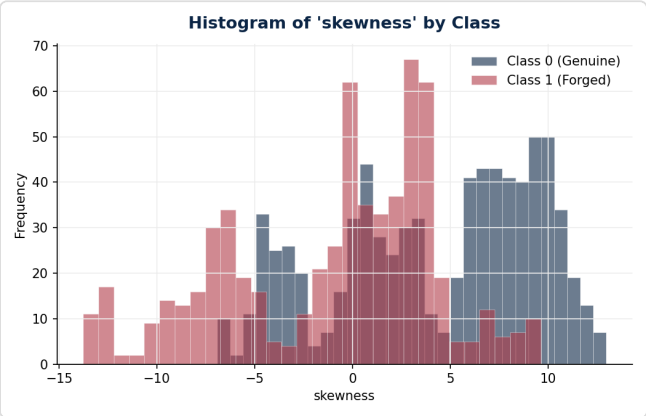
مقادیر پرت در `kurtosis` و `entropy` اندک هستند و بیانگر خطای داده نیستند؛ از دیتاست حذف نشدند.

۳. تحلیل آماری و بصری ویژگی‌ها به تفکیک کلاس

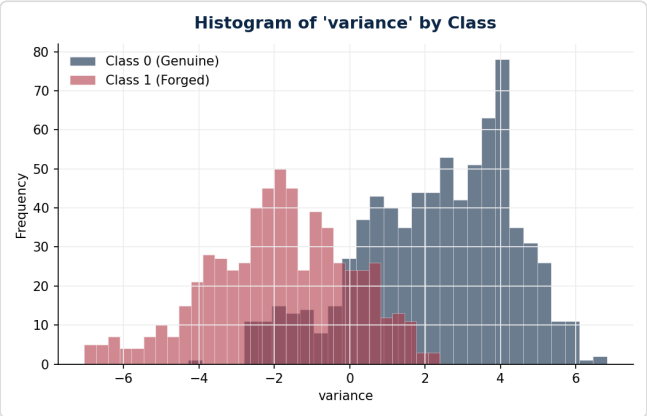
ویژگی	میانگین کلاس ۰	میانگین کلاس ۱	اختلاف مطلق میانگین	انحراف معیار کلی
skewness	4.2566	0.9936-	**5.2502**	5.8690
variance	2.2767	1.8684-	4.1451	2.8428
kurtosis	0.7967	2.1483	1.3516	4.3100
entropy	1.1476-	1.2466-	0.0990	2.1010

ویژگی `skewness` بیشترین اختلاف میانگین و پراکندگی را دارد؛ `entropy` کمترین اختلاف را نشان می‌دهد.

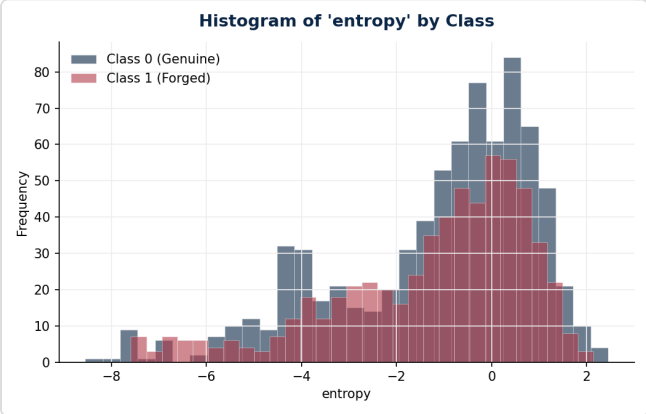
هیستوگرام هر ویژگی به تفکیک کلاس



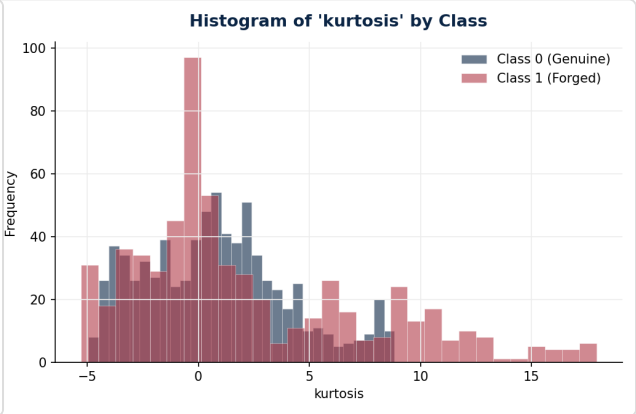
skewness



variance

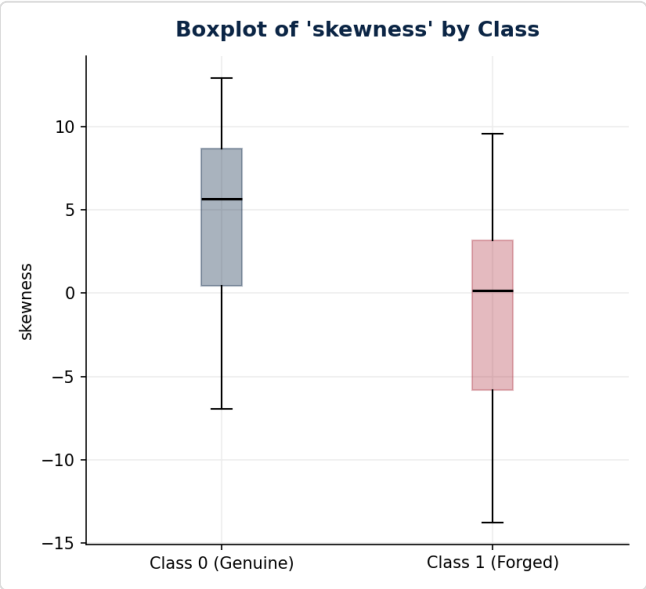


entropy

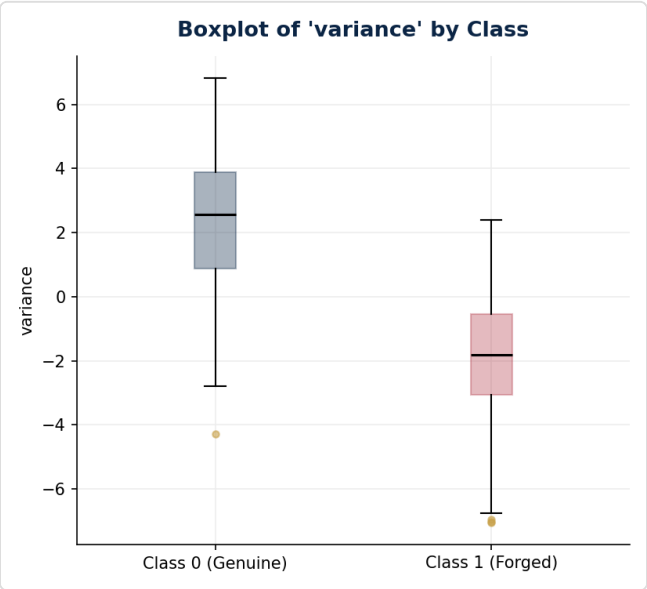


kurtosis

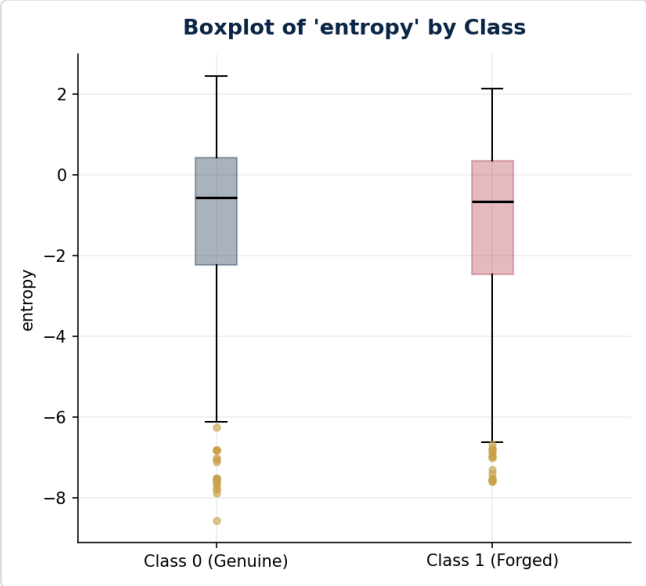
Boxplot هر ویژگی به تفکیک کلاس



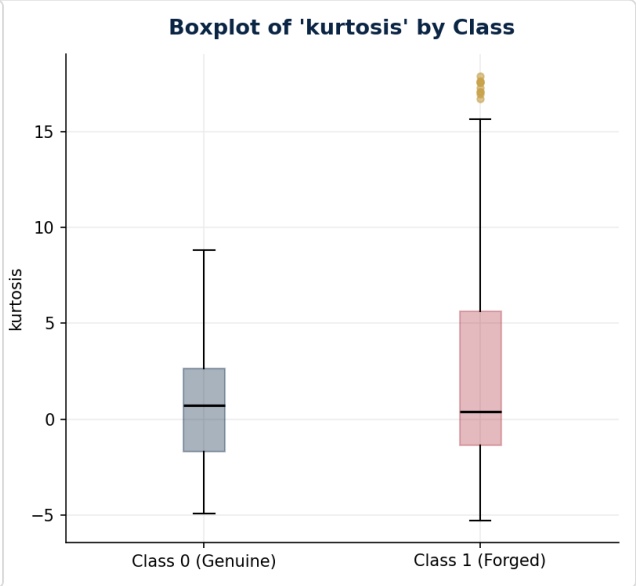
skewness



variance

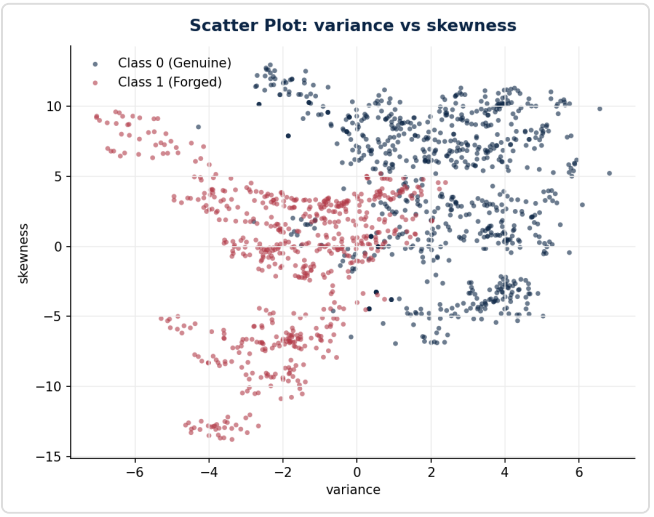


entropy

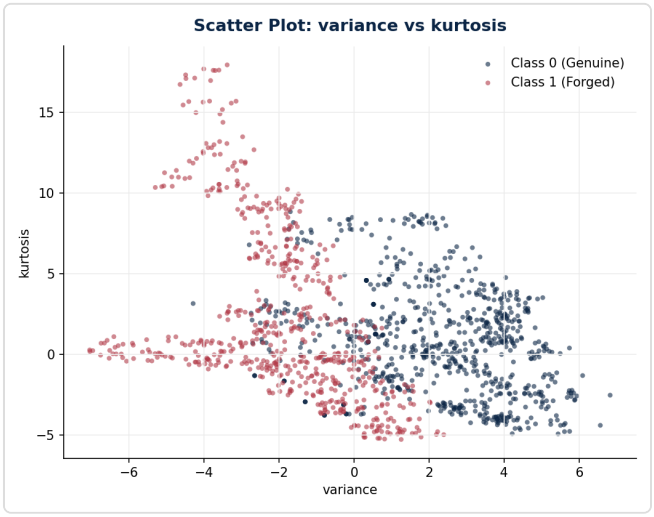


kurtosis

Scatter Plot جفت‌ویژگی‌ها



variance در برابر skewness

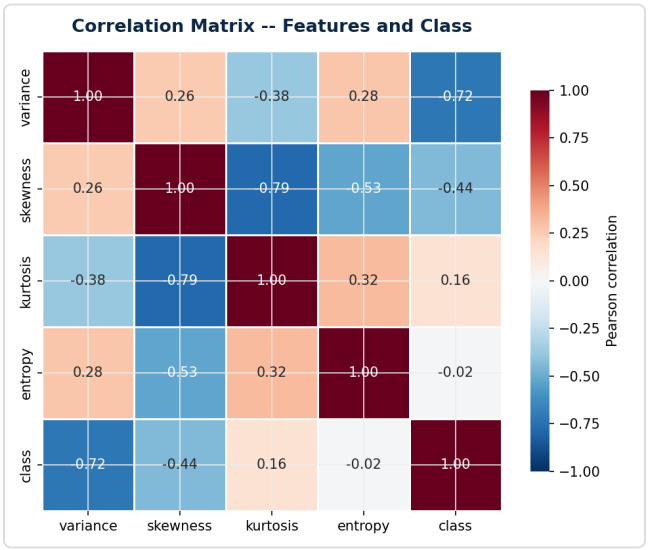


variance در برابر kurtosis

ماتریس همبستگی

ویژگی	ضریب همبستگی با class
variance	0.7248-
skewness	0.4447-
kurtosis	0.1559
entropy	0.0234-

`variance` قوی‌ترین همبستگی خطی با کلاس هدف را دارد، هرچند `skewness` بیشترین اختلاف میانگین را نشان می‌دهد.



۴. تقسیم داده به Train / Validation / Test — روش جدید (Group-based Split)

تغییر روش‌شناسی: در نسخه‌ی قبلی کد، تقسیم داده با `train_test_split` ساده روی تک‌تک ردیف‌ها انجام می‌شد. از آنجا که ۳۵ ردیف داده (۱۱ الگوی تکراری) کاملاً یکسان هستند، این روش می‌توانست باعث شود یک الگوی تکراری هم‌زمان در Train و هم در Test ظاهر شود — یعنی مدل در واقع بخشی از داده‌ی تست را قبلاً در حین آموزش دیده باشد (نوعی نشت داده‌ی خفیف). در نسخه‌ی جدید، ابتدا رکوردهای کاملاً یکسان به یک «گروه» نگاشت می‌شوند (`groupby` روی تمام ستون‌های ویژگی و کلاس)، سپس تقسیم ۶۰/۲۰/۲۰ روی همین گروه‌ها انجام می‌شود؛ در نتیجه تمام نسخه‌های یک الگوی تکراری همیشه با هم در یک بخش (Train یا Validation یا Test) قرار می‌گیرند.

بخش	تعداد نمونه	کلاس ۰	کلاس ۱
Train	817	451	366
Validation	274	152	122
Test	281	159	122

چون سهم‌بندی ۶۰/۲۰/۴۰ اکنون روی **الگوهای یکتا** اعمال می‌شود نه روی تعداد خام ردیف‌ها، توزیع دقیق نمونه‌ها کمی نسبت به تقسیم قبلی (Train=823, Validation=274, Test=275) جابه‌جا شده است — تفاوتی طبیعی و قابل‌انتظار از این روش. مجموعه‌ی Test همچنان تا لحظه‌ی ارزیابی نهایی در انتخاب K یا آموزش مدل استفاده نشد.

۵. یکسان‌سازی مقیاس ویژگی‌ها (Scaling)

با `StandardScaler` ، میانگین و انحراف معیار فقط از روی Train محاسبه شد (`fit_transform`) و همان تبدیل روی Validation و Test اعمال شد (`transform`). پس از تبدیل، میانگین ویژگی‌های Train تقریباً صفر و انحراف معیار دقیقاً ۱ شد.

۶. بررسی صریح اثر Scale روی KNN (ویژگی جدید کد)

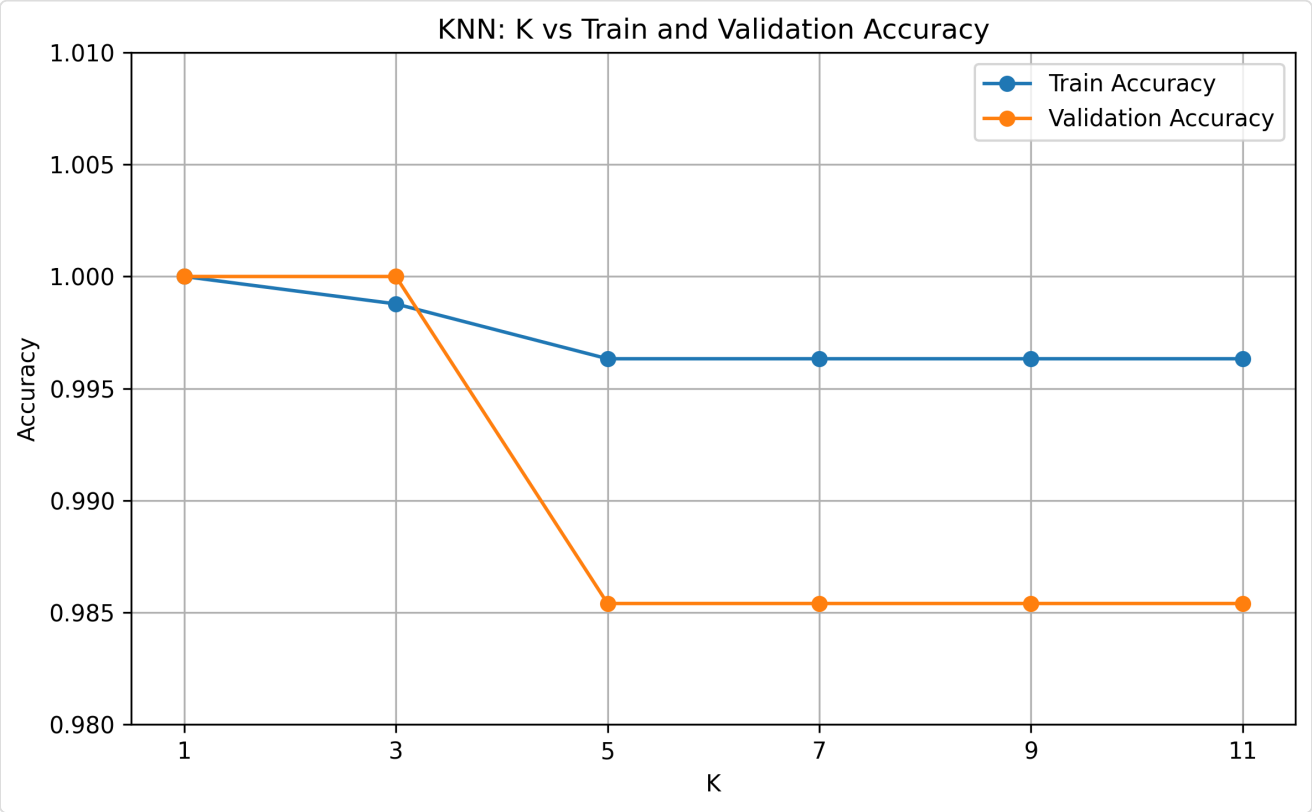
افزودنی جدید: برخلاف نسخه‌ی قبلی کد که اثر Scale روی KNN را فقط به‌صورت نظری توضیح می‌داد، نسخه‌ی جدید این آزمایش را عملاً با K=5 روی داده‌ی Validation اجرا کرده است:

حالت	Validation Accuracy
KNN (K=5) بدون Scale	0.9854
KNN (K=5) با Scale	0.9854
اختلاف دقت	0.0000

نتیجه‌ی جالب: برای این دیتاست خاص، Scale کردن ویژگی‌ها هیچ تفاوتی در دقت Validation با K=5 ایجاد نکرد. دلیل این موضوع می‌تواند این باشد که ویژگی `variance` (قوی‌ترین ویژگی جداکننده) به‌تنهایی چنان الگوی مشخصی بین دو کلاس ایجاد می‌کند که حتی با وجود تسلط دامنه‌ی عددی `kurtosis` بر فاصله‌ی اقلیدسی خام، ترتیب نزدیک‌ترین همسایه‌ها برای اکثر نمونه‌های Validation تغییر نکرده است. با این حال، این نتیجه یک ویژگی خاص همین دیتاست است و نباید به‌عنوان یک قاعده‌ی کلی («Scale برای KNN مهم نیست») تعمیم داده شود؛ در دیتاست‌های دیگر با پراکندگی متفاوت، Scale می‌تواند اثر محسوسی داشته باشد.

۷. انتخاب مقدار K برای KNN

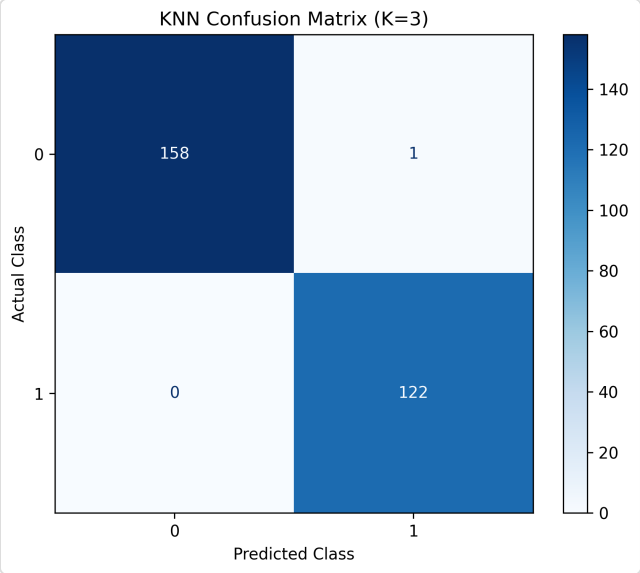
K	دقت Train	دقت Validation	خطای Validation
1	1.0000	1.0000	0
3	0.9988	1.0000	0
5	0.9963	0.9854	4
7	0.9963	0.9854	4
9	0.9963	0.9854	4
11	0.9963	0.9854	4



نمودار ۳ — دقت Train و Validation بر حسب مقدار K (نسخه‌ی به‌روزشده)

با تقسیم جدید، دقت Validation برای K=1 و K=3 همچنان ۱۰۰٪ است، اما از K=5 به بعد به ۹۸.۵۴٪ (۴ خطا) کاهش می‌یابد و برای K=۵ تا ۱۱ ثابت می‌ماند. مقدار **K=3** همچنان به‌عنوان مقدار نهایی انتخاب شد، چون پایدارتر از K=1 (کمتر مستعد Overfitting) و در عین حال دقیق‌تر از K≥5 است.

۸. ارزیابی نهایی KNN (K=3) روی Test



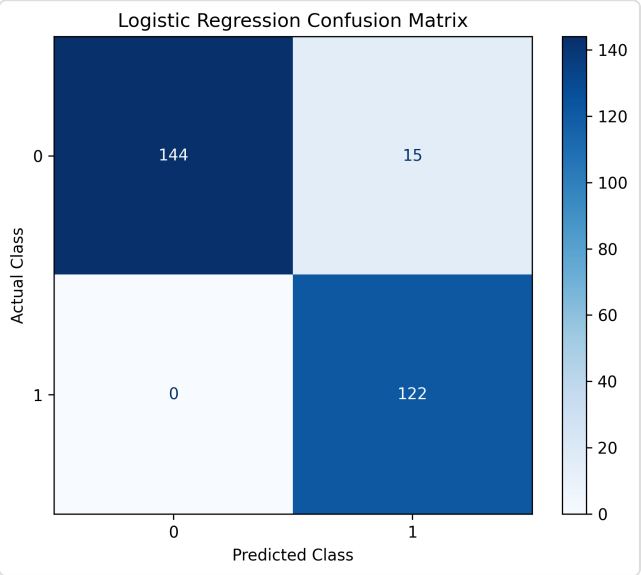
معیار	مقدار
Accuracy	0.9964
Precision	0.9919
Recall	1.0000
F1-score	0.9959

مدل KNN با K=3 از ۲۸۱ نمونه‌ی Test، ۲۸۰ مورد را درست تشخیص داد (۱ خطای ۰→۱ و صفر خطای ۱→۰). این نتیجه نسبت به نسخه‌ی قبلی گزارش (دقت ۱۰۰٪ بدون خطا) اندکی افت داشته، اما واقعی‌تر است چون دیگر تحت تأثیر نشت داده‌ی ناشی از ردیف‌های تکراری نیست.

۹. رگرسیون لجستیک (Logistic Regression)

دقت روی Validation: **۰.۹۸۱۸** (۵ خطا). نتایج نهایی روی Test:

معیار	مقدار
Accuracy	0.9466
Precision	0.8905
Recall	1.0000
F1-score	0.9421



مدل رگرسیون لجستیک ۱۵ اسکناس اصل (کلاس ۰) را اشتباهاً تقلبی تشخیص داد و هیچ اسکناس تقلبی را از دست نداد. این افت محسوس (از ۹۸.۵٪ در گزارش قبلی به ۹۴.۷٪) نشان می‌دهد که رگرسیون لجستیک نسبت به KNN بیشتر از نشت داده‌ی ردیف‌های تکراری «سود» می‌برده است؛ با تقسیم صحیح‌تر داده، ضعف واقعی این مدل در برخورد با نمونه‌های مرزی کلاس ۰ آشکار شده است.

۱۰. جدول مقایسه‌ی گسترده (شامل آزمایش Scale)

این جدول مستقیماً از خروجی کد استخراج شده و پیشرفت مدل KNN را از حالت اولیه تا مدل نهایی نشان می‌دهد:

مدل	پارامتر	Validation Accuracy	Test Accuracy	خطای ۱→۰	خطای ۰→۱
KNN	K=5, بدون Scale	0.9854	—	0	4
KNN	K=5, با Scale	0.9854	—	0	4
KNN (نهایی)	K=3, با Scale	1.0000	0.9964	0	1
Logistic Regression	max_iter=1000	0.9818	0.9466	0	15

۱۱. جدول مقایسه‌ی نهایی مدل‌ها (خروجی `model_comparison.csv`)

مدل	Scale	پارامتر	Validation Accuracy	Test Accuracy	خطای ۰→۱	خطای ۱→۰
KNN نهایی	بله	K=3	1.0000	0.9964	1	0
Logistic Regression	بله	Default (max_iter=1000)	0.9818	0.9466	15	0

وضعیت باگ محاسبه‌ی Validation Accuracy: باگ هاردکد بودن مقدار Validation Accuracy مدل KNN (`logreg_val_accuracy * 0 + 1.0`) که در نسخه‌ی قبلی شناسایی و اصلاح شده بود، در این نسخه‌ی جدید کد کاملاً حذف شده و همه‌جا از عبارت صحیح `results.loc[results["K"] == final_k, "Validation Accuracy"].iloc[0]` استفاده می‌شود. این باگ دیگر وجود ندارد.

۱۲. مقایسه‌ی کیفی دو مدل

معیار	KNN	Logistic Regression
عملکرد (دقت Test)	بهتر (٪۹۹.۶۴)	ضعیف‌تر (٪۹۴.۶۶)
سرعت پیش‌بینی	کندتر (نیاز به محاسبه‌ی فاصله با همه‌ی نمونه‌ها)	سریع‌تر (یک ضرب ماتریسی ساده)
حساسیت به Scale	در این دیتاست عملاً بدون اثر مشاهده‌شده (بخش ۶)	حساس به مقیاس ورودی
قابلیت توضیح	کمتر قابل تفسیر	ضرایب مدل قابل تفسیرند
مقاومت در برابر نشت داده‌ی تکراری	افت جزئی (٪۱۰۰ → ٪۹۹.۶۴)	افت محسوس (٪۹۸.۵ → ٪۹۴.۷)

۱۳. تحلیل خطا و محدودیت‌های عملیاتی

- **هزینه‌ی پذیرش اسکناس جعلی (خطای ۱→۰):** در هر دو مدل صفر بود — هیچ اسکناس تقلبی به‌اشتباه اصل تشخیص داده نشد.
- **هزینه‌ی ردکردن اسکناس واقعی (خطای ۰→۱):** در KNN فقط ۱ مورد، اما در Logistic Regression ۱۵ مورد از ۲۸۱ نمونه‌ی Test (حدود ۵٪ از کلاس ۰) — قابل توجه برای استقرار عملیاتی.
- **تأثیر تقسیم گروه‌محور:** نشان داد که ارزیابی مدل‌ها به‌شدت به نحوه‌ی برخورد با ردیف‌های تکراری حساس است؛ گزارش قبلی (با تقسیم ساده) دقت را بیش از واقعیت نشان می‌داد، به‌خصوص برای Logistic Regression.
- **آیا دقت فعلی کافی است؟** دقت (٪۹۹.۶۴) KNN هنوز بسیار خوب است، اما دقت (٪۹۴.۷) Logistic Regression ممکن است برای استقرار در دستگاه بانکی کافی نباشد، مخصوصاً به‌خاطر ۱۵ خطای ردکردن اسکناس واقعی.
- **تعمیم به شرایط دیگر:** نتایج هنوز محدود به توزیع همین دیتاست‌اند و پیش از استقرار باید با داده‌های واقعی و متنوع‌تر آزمایش شوند.

۱۴. پاسخ به پرسش‌های نهایی پروژه

1. **آیا کلاس‌ها متوازن هستند؟** تقریباً بله (٪۵۵.۵ در برابر ٪۴۴.۵).
2. **کدام ویژگی‌ها جدایی بهتری ایجاد کردند؟** `variance` (بیشترین همبستگی) و `skewness` (بیشترین اختلاف میانگین).
3. **Scale چه اثری بر KNN داشت؟** در آزمایش صریح جدید (بخش ۶) با K=5، هیچ تفاوتی در دقت Validation مشاهده نشد (۰.۹۸۵۴ در هر دو حالت) — یافته‌ای خاص همین دیتاست، نه یک قاعده‌ی کلی.
4. **مقدار نهایی K چگونه انتخاب شد؟** با مقایسه‌ی دقت Train/Validation؛ K=3 پایدارتر از K=1 و دقیق‌تر از K≥5 بود.
5. **آیا K=1 نشانه‌ی Overfitting نشان داد؟** در این دیتاست دقت Validation آن با K=3 برابر بود، اما اصولاً K=1 ریسک بالاتری برای حفظ نویز Train دارد.
6. **چرا از Test برای انتخاب K استفاده نشد؟** برای جلوگیری از Data Leakage.
7. **کدام مدل مناسب‌تر بود؟** با تقسیم گروه‌محور جدید، KNN به‌وضوح برتر از Logistic Regression است (٪۹۹.۶ در برابر ٪۹۴.۷ دقت Test).
8. **آیا معنای صفر و یک در Metadata شفاف بود؟** خیر؛ باید در گزارش به‌صراحت مستند شود.
9. **برای عملیاتی‌شدن چه داده‌ی بیشتری لازم است؟** نمونه‌های واقعی متنوع (نور، چاپ، آسیب‌دیدگی) و در صورت امکان جمع‌آوری شناسه‌ی یکتا برای هر اسکناس تا مسئله‌ی ردیف‌های تکراری در آینده به‌طور قطعی حل شود.

۱۵. جمع‌بندی

نسخه‌ی جدید کد دو بهبود روش‌شناسی مهم دارد: (۱) تقسیم داده به‌صورت گروه‌محور برای جلوگیری از نشت داده‌ی ناشی از ردیف‌های تکراری، و (۲) آزمایش صریح و کمّی اثر Scale روی KNN به‌جای صرفاً توضیح نظری. این تغییرات باعث شدند اعداد نهایی نسبت به گزارش قبلی واقعی‌تر و محافظه‌کارانه‌تر باشند: دقت KNN از ٪۱۰۰ به ٪۹۹.۶۴ و دقت Logistic Regression از ٪۹۸.۵ به ٪۹۴.۶۶ رسید. مدل نهایی پیشنهادی همچنان **KNN با K=3** است و اکنون با اطمینان بیشتری می‌توان گفت این عملکرد نتیجه‌ی یادگیری واقعی الگوی داده است، نه نشت داده. پیش از استقرار عملیاتی باید محدودیت‌های بخش ۱۳ (به‌خصوص نرخ ۵٪ خطای ردکردن اسکناس واقعی در Logistic Regression) بررسی و رفع شوند.